📝 Part 4 — Short Answers

Author: Rozagul Nodirbekova · **Date:** 2026-08-10

---

1. Difference between Smoke and Regression testing?

Smoke testing is a quick, shallow check run right after a new build to confirm the critical paths work at all — "is the build stable enough to test?" (e.g. app launches, login succeeds, a restaurant opens). **Regression testing** is a broad, deep re-run of existing test cases after a change to confirm that *previously working* features still work and the change introduced no side effects. In short: smoke is **wide-but-shallow and fast** (a gate), regression is **narrow-but-deep and thorough** (a safety net), and smoke usually runs *before* regression.

---

2. Why use Explicit Waits instead of `Thread.sleep()`?

`Thread.sleep()` blocks for a **fixed** duration regardless of the app state, so it is both slow (you always pay the full wait even when the element is ready instantly) and flaky (a device that is briefly slower than your guess still fails). An **explicit wait** (e.g. `WebDriverWait … until(expected_conditions)`) polls for a **specific condition** — element visible, clickable, text present — and continues the moment it is true, up to a timeout. This makes tests **faster and far more reliable**. `Thread.sleep()` should only appear as a rare, commented last resort (e.g. waiting on a non-observable animation).

---

3. Three key differences between mobile and web testing?

1. **Environment fragmentation & gestures** — mobile must cover many device sizes, OS versions, and OEM skins, plus touch gestures (tap, long-press, swipe, pinch), whereas web centres on browsers, viewports and mouse/keyboard.
2. **Interruptions & device state** — mobile apps must survive incoming calls, notifications, backgrounding, low battery, permission dialogs, and OS memory-kills; the web rarely deals with these.
3. **Network & sensors** — mobile is tested across flaky/edge networks (3G/offline), and uses GPS, camera, and push notifications; installation happens via app stores rather than a URL.

---

4. A developer says "not a bug, expected behavior" — what do you do?

I stay factual and collaborative rather than defensive. I re-check the **requirement / spec / acceptance criteria** and design and confirm my reproduction steps, environment and evidence (screenshot, video, logs). Then I bring **data**: "Here is the AC / user story it violates, here are exact steps and the impact on the user." If it genuinely isn't covered by any spec, it's a **requirements gap**, so I escalate to the PO/BA to get a decision and document the outcome — we either fix it, or consciously accept it and update the spec so it's no longer ambiguous. The goal is the right product decision, not winning an argument.

---

5. Three mobile-specific issues to watch for in a food delivery app?

1. **Location & maps accuracy** — wrong/denied GPS permission, stale address, or map pin drift sends orders to the wrong place; test permission flows, manual address entry, and geofencing of delivery zones.
2. **Network resilience during ordering & payment** — the user may lose signal mid-checkout; the app must not double-charge, double-submit, or lose the cart, and must handle payment timeouts idempotently.

3. **Real-time order tracking, notifications & app lifecycle** — live driver tracking and status push notifications must keep working when the app is backgrounded or the screen is locked, without draining the battery or showing stale ETAs.